

COMP 3105A Introduction to Machine Learning

Assignment 1

Instructor: Junfeng Wen (junfeng.wen [AT] carleton.ca)

Fall 2026
School of Computer Science
Carleton University

Deadline: 11:59 pm, Sunday, Oct. 4, 2026

Instructions: Submit the following three files to Brightspace for marking

- A Python file `A1codes.py` that includes all your implementations. **Other than imports, constant assignments, and function definitions, do not include top-level statements in this file. In particular, do not call functions, load datasets, run experiments, or print output when the file is imported.**
- A pip [requirements file](#) named `requirements.txt` that specifies the running environment including a list of Python libraries/packages and their versions required to run your code. The current template uses Python 3.10, which will be used in the grading environment.
- A PDF file `A1report.pdf` that includes all your answers to the written questions. It should also specify your team members (names and student IDs). Please clearly specify question/sub-question numbers in your submitted PDF report so TAs can see which question you are answering.

Do not submit a compressed file, or it will result in a mark deduction. We recommend trying your code using Colab or Anaconda/Virtualenv before submission.

Rubrics: This assignment is worth 15% of the final grade. Your code and report will be evaluated based on their scientific qualities including but not limited to: Are the implementations correct? Is the analysis rigorous and thorough? Are the code easily understandable (with comments)? Is the report well-organized and clear?

Policies:

- You can finish this assignment in groups of two. All members of a group will receive the same mark when the workload is shared.
- You can resubmit your work on Brightspace and the old submissions will be overwritten. However, please **submit all required files** (as opposed to only submitting the file that you want to overwrite) on Brightspace as we will only evaluate your latest submission.
- You may consult others (classmates/TAs) about general ideas but don't share codes/answers. Please specify in the PDF file any individuals you consult for the assignment. Any student found to cheat or violate this policy will receive a score of 0 for this assignment.

- You can **only** ask large language models (LLMs) general functionality questions. For example, “How do I remove all negative numbers in a NumPy array?” If you used LLMs, clearly show us how you use them (either include the prompts and replies in the report, or preferably include URL links showing chat history in the report). Any student found to cheat or violate this policy will receive a score of 0 for this assignment.
 - Remember that you have **three** excused days *throughout the term* (rounded up to the nearest day), after which no late submission will be accepted.
 - Specifically for this assignment, you can use libraries with general utilities, such as matplotlib, numpy/scipy, cvxopt, and pandas for Python. **However, you must implement everything by yourselves without using any pre-existing implementations of the algorithms or any functions from an ML library (such as scikit-learn).** The goal is for you to really understand, step by step, how the algorithms work.
-

Question 1 (7.5%) Linear Regression

In this question, you will implement linear regression from scratch, in Python using NumPy/SciPy, and evaluate their performance on different datasets. You will learn the basics of array manipulations and matrix/vector operations (e.g., use `@` for matrix multiplication, `X.T` to transpose a matrix `X` etc). You will also learn some essential functions like `numpy.linalg.solve` to solve linear systems and `cvxopt.solvers.lp` to solve linear programming problems.

All of the following functions must be able to handle arbitrary $n > 0$ and $d > 0$. The vectors and matrices are represented as NumPy arrays. Your functions shouldn't print additional information to the standard output.

(a) (0.5%) L_2 Regression

Implement a Python function

$$\mathbf{w} = \text{minimizeL2}(\mathbf{X}, \mathbf{y})$$

that takes an $n \times d$ input matrix `X` and an $n \times 1$ target/label vector `y`, and returns a $d \times 1$ vector of weights/parameters `w` corresponding to the solution of the L_2 losses:

$$\mathbf{w} = \underset{\mathbf{w} \in \mathbb{R}^d}{\operatorname{argmin}} \frac{1}{2n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (1)$$

Use the analytic solution above for your implementation. Here we assume that the inverse exists.

Hint: You may find `np.linalg.solve` or `np.linalg.inv` helpful.

(b) (2%) L_1 Regression

Here we are going to solve the L_1 loss regression problem

$$\mathbf{w} = \underset{\mathbf{w} \in \mathbb{R}^d}{\operatorname{argmin}} \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_1.$$

Recall that the optimization can be expressed as linear programming with the joint parameters $\begin{bmatrix} \mathbf{w} \\ \boldsymbol{\delta} \end{bmatrix} \in \mathbb{R}^{d+n}$ as follows

$$\begin{aligned} \min_{\mathbf{w}, \boldsymbol{\delta}} \quad & \boldsymbol{\delta}^\top \mathbf{1}_n \\ \text{s.t.} \quad & \boldsymbol{\delta} \succeq \mathbf{0}_n \\ & \mathbf{X}\mathbf{w} - \mathbf{y} \preceq \boldsymbol{\delta} \\ & \mathbf{y} - \mathbf{X}\mathbf{w} \preceq \boldsymbol{\delta}. \end{aligned}$$

Similar to the input/output specifications of the previous question, we would like to implement a Python function

$$\mathbf{w} = \text{minimizeL1}(\mathbf{X}, \mathbf{y})$$

that returns a $d \times 1$ vector of weights/parameters `w` corresponding to the solution.

(b.1) (1%) Suppose that someone asked generative AI to solve it and it generates the following code:

```
1 def minimizeL1(X, y):
2     # first d variables are w, second n variables are delta
3     n, d = X.shape
4
5     c = np.concatenate([np.zeros(n), np.ones(d)])
6
7     G = np.concatenate([np.concatenate([np.zeros_like(X), -np.eye(n)], axis=1),
```

```

8         np.concatenate([X, -np.eye(n)], axis=1),
9         np.concatenate([X, -np.eye(n)], axis=1)
10    ])
11
12    h = np.concatenate([np.zeros_like(y), y, -y])
13
14    cvxopt.solvers.options["show_progress"] = False
15    res = cvxopt.solvers.lp(c, G, h)
16    w = res['x'][-d:]
17    return np.array(w)

```

Assume the following

- the three row blocks of G and h are intended to correspond, in order, to $\delta \succeq \mathbf{0}_n$, $X\mathbf{w} - \mathbf{y} \preceq \delta$, and $\mathbf{y} - X\mathbf{w} \preceq \delta$;
- `numpy` as `np` and `cvxopt` are already imported;
- failure-status and input-validation issues are outside scope;
- omission of the $1/n$ objective scaling does not count as a bug.

Without adding or deleting any line of code in the above, write down all bugs and their corresponding line numbers in the PDF report. Explain why they are wrong. Here is an example:

- Line 15: The input matrices should be `cvxopt.matrix` instead of NumPy arrays, because `cvxopt.solvers.lp` only accepts `cvxopt.matrix` as inputs.

There are **exactly four intended bugs** in the code, including the one above. Follow the exact format:

- Line [XX]: [BUG], because [REASON].

(b.2) (1%) With the bugs identified, write your correct implementation in the submitted file `A1codes.py`.

(c) (2%) L_∞ Regression

Here we are going to solve the L_∞ loss regression problem

$$\mathbf{w} = \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^d} \|X\mathbf{w} - \mathbf{y}\|_\infty.$$

Recall that this optimization can be expressed as a linear program with the joint parameters $\begin{bmatrix} \mathbf{w} \\ \delta \end{bmatrix} \in \mathbb{R}^{d+1}$ as follows

$$\begin{aligned} \min_{\mathbf{w}, \delta} \quad & \delta \\ \text{s.t.} \quad & \delta \geq 0 \\ & X\mathbf{w} - \mathbf{y} \preceq \delta \cdot \mathbf{1}_n \\ & \mathbf{y} - X\mathbf{w} \preceq \delta \cdot \mathbf{1}_n \end{aligned} \tag{2}$$

Now we want to convert it into a form that is solvable by the `cvxopt` linear programming (LP) solver, which solves the following form of LP

$$\begin{aligned} \min_{\mathbf{u}} \quad & \mathbf{c}^\top \mathbf{u} \\ \text{s.t.} \quad & G\mathbf{u} \preceq \mathbf{h}. \end{aligned}$$

Let the unknown variables be $\mathbf{u} = \begin{bmatrix} \mathbf{w} \\ \delta \end{bmatrix} \in \mathbb{R}^{d+1}$.

For the constraints, since we have three sets of constraints, the matrix G and $\mathbf{h}$ can be decomposed into three parts

$$G \cdot \mathbf{u} = \begin{bmatrix} G^{(1)} \\ G^{(2)} \\ G^{(3)} \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ \delta \end{bmatrix} \preceq \begin{bmatrix} \mathbf{h}^{(1)} \\ \mathbf{h}^{(2)} \\ \mathbf{h}^{(3)} \end{bmatrix} = \mathbf{h}$$

and each part corresponds to a constraint specified in Eq. (2).

Write your answers to the following questions in the PDF report. **Please specify the shape of each part of your expression clearly.**

(c.1) (0.25%) For the objective function, we want $\mathbf{c}^\top \mathbf{u} = \delta$. What should $\mathbf{c} \in \mathbb{R}^{d+1}$ be?

(c.2) (0.25%) We want $G^{(1)}\mathbf{u} \preceq \mathbf{h}^{(1)} \iff \delta \geq 0$. What should $G^{(1)} \in \mathbb{R}^{1 \times (d+1)}$ and $\mathbf{h}^{(1)} \in \mathbb{R}$ be?

(c.3) (0.25%) We want $G^{(2)}\mathbf{u} \preceq \mathbf{h}^{(2)} \iff X\mathbf{w} - \mathbf{y} \preceq \delta \cdot \mathbf{1}_n$. What should $G^{(2)} \in \mathbb{R}^{n \times (d+1)}$ and $\mathbf{h}^{(2)} \in \mathbb{R}^n$ be?

(c.4) (0.25%) We want $G^{(3)}\mathbf{u} \preceq \mathbf{h}^{(3)} \iff \mathbf{y} - X\mathbf{w} \preceq \delta \cdot \mathbf{1}_n$. What should $G^{(3)} \in \mathbb{R}^{n \times (d+1)}$ and $\mathbf{h}^{(3)} \in \mathbb{R}^n$ be?

(c.5) (1%) Based on your derivations above, implement a Python function

```
w = minimizeLinf(X, y)
```

that returns a $d \times 1$ vector of weights/parameters $\mathbf{w}$ corresponding to the solution of minimum L_∞ loss. Note that if you do not complete part (c.1) - (c.4), you won't receive marks for part (c.5).

Hints: You may find `np.zeros`, `np.ones`, `np.concatenate` and `cvxopt.solvers.lp` helpful. Make sure all matrices/vectors are converted to `cvxopt.matrix` first before calling the solver. You should also set `solvers.options['show_progress'] = False` to silence the solver in your final submission. After finishing parts (a) to (c), you can use `Alttestbed.py` to visualize your models and make sure they are reasonable.

(d) (1.5%) Synthetic Regression Problem

In this part, you will evaluate your implemented algorithms on a synthetic dataset.

(d.1) (0.5%) Implement a Python function

```
train_loss, test_loss = synRegExperiments()
```

that returns a 3×3 matrix `train_loss` of *average* training losses and a 3×3 matrix `test_loss` of *average* test losses (See Table 1 and Table 2 below.) It repeats 50 runs as follows

```
def synRegExperiments():
```

```
    def genData(n_points, is_training=False):
```

```
        '''
```

```
        This function generates synthetic data
```

```
        '''
```

```
        X = np.random.randn(n_points, d) # input matrix
```

```
        X = np.concatenate((np.ones((n_points, 1)), X), axis=1) # augment input
```

```
        y = X @ w_true + np.random.randn(n_points, 1) * noise # ground truth label
```

```
        if is_training:
```

```

        y[0] *= -0.1
    return X, y

n_runs = 50
n_train = 30
n_test = 1000
d = 5
noise = 0.2
train_loss = np.zeros([n_runs, 3, 3]) # n_runs * n_models * n_metrics
test_loss = np.zeros([n_runs, 3, 3]) # n_runs * n_models * n_metrics

# TODO: Change the following random seed to one of your student IDs
np.random.seed(42)

for r in range(n_runs):

    w_true = np.random.randn(d + 1, 1)
    Xtrain, ytrain = genData(n_train, is_training=True)
    Xtest, ytest = genData(n_test, is_training=False)

    w_L2 = minimizeL2(Xtrain, ytrain)
    w_L1 = minimizeL1(Xtrain, ytrain)
    w_Linf = minimizeLinf(Xtrain, ytrain)

    # TODO: Evaluate the three models' performance (for each model,
    #         calculate the L2, L1 and L infinity losses on the training
    #         data). Save them to `train_loss`

    # TODO: Evaluate the three models' performance (for each model,
    #         calculate the L2, L1 and L infinity losses on the test
    #         data). Save them to `test_loss`

# TODO: compute the average losses over runs
# TODO: return a 3-by-3 training loss variable and a 3-by-3 test loss variable

```

Note that the L_1 and L_2 losses should be the average loss over training/test points.

In the PDF report, show the *averages* (over 50 runs) for each kind of loss and each kind of model in two tables below.

Table 1: Different training losses for different models

Model	L_2 loss	L_1 loss	L_∞ loss
L_2 model	(enter training losses)		
L_1 model			
L_∞ model			

Table 2: Different test losses for different models

Model	L_2 loss	L_1 loss	L_∞ loss
L_2 model	(enter test losses)		
L_1 model			
L_∞ model			

Debug Hint: For your verification, we include some toy data in the `toy_data` folder. If you train with the `regression_train.csv` (with augmentation), you should get the following models

$$\mathbf{w}_{L2} = \begin{bmatrix} -26.72481802 \\ -1.1663904 \end{bmatrix} \quad \mathbf{w}_{L1} = \begin{bmatrix} -26.88576639 \\ -1.16315391 \end{bmatrix} \quad \mathbf{w}_{L\infty} = \begin{bmatrix} -25.3365398 \\ -1.39556938 \end{bmatrix}$$

with the following training and test losses:

Table 3: Different training losses for different models for `regression_train.csv`

Model	L_2 loss	L_1 loss	L_∞ loss
L_2 model	2.00305568	1.56314076	7.31711154
L_1 model	2.01278477	1.55829245	7.4682775
L_∞ model	2.36347873	1.72061687	6.6215357

Table 4: Different test losses for different models for the `regression_test.csv`

Model	L_2 loss	L_1 loss	L_∞ loss
L_2 model	1.66071948	1.42055149	5.58874693
L_1 model	1.62892803	1.40563309	5.45534197
L_∞ model	1.9868478	1.58480749	5.02664782

(d.2) (1%) Looking at your tables from above, analyze the results and discuss any findings you may have and the possible reason behind them.

(e) (1.5%) Real-World Regression Problem

(e.1) (1%) Here you will apply the linear regression algorithms to the [concrete compressive strength](#) (CCS) dataset (click to see the data website).

To start, you need to preprocess the data. Implement a Python function

```
X, y = preprocessCCS(dataset_folder)
```

that takes an absolute path (the result of `os.path.abspath`) of the CCS `dataset_folder` (where the `ConcreteData.xls` file is located) and returns the preprocessed $n \times d$ input matrix `X` with raw features and an $n \times 1$ label vector `y` (which are suitable for use with your functions in (a) to (c)).

(e.2) (0.5%) Implement a Python function

```
train_loss, test_loss = runCCS(dataset_folder)
```

that takes the absolute dataset path and returns the training and test losses. It runs as follows

```
def runCCS(dataset_folder):

    X, y = preprocessCCS(dataset_folder)
    n, d = X.shape
    X = np.concatenate((np.ones((n, 1)), X), axis=1) # augment

    n_runs = 50
    train_loss = np.zeros([n_runs, 3, 3]) # n_runs * n_models * n_metrics
    test_loss = np.zeros([n_runs, 3, 3]) # n_runs * n_models * n_metrics

    # TODO: Change the following random seed to one of your student IDs
    np.random.seed(42)

    for r in range(n_runs):

        # TODO: Randomly partition the dataset into two parts (50%
        #         training and 50% test)

        # TODO: Learn three different models from the training data
        #         using L2, L1 and L infinity losses

        # TODO: Evaluate the three models' performance (for each model,
        #         calculate the L2, L1 and L infinity losses on the training
```

```
#         data). Save them to `train_loss`

# TODO: Evaluate the three models' performance (for each model,
#         calculate the L2, L1 and L infinity losses on the test
#         data). Save them to `test_loss`

# TODO: compute the average losses over runs
# TODO: return a 3-by-3 training loss variable and a 3-by-3 test loss variable
```

In the PDF file, report the *averages* (over 50 runs) for each kind of loss and each kind of model using tables similar to Table 1 and Table 2.

Question 2 (7.5%) Logistic Regression

In this question, you will implement logistic regression, a classification method, and solve it using `scipy.optimize.minimize`.

All of the following functions must be able to handle arbitrary $n > 0$ and $d > 0$. The vectors and matrices are represented as NumPy arrays. Your functions shouldn't print additional information to the standard output.

(a) (2%) Solving Linear Regression (Again)

(a.1) (1%) Before diving into logistic regression, let's first revisit the linear regression in Q1(a). Implement two Python functions

```
obj_val = linearRegL2Obj(w, X, y)
gradient = linearRegL2Grad(w, X, y)
```

that take a $d \times 1$ vector of parameters $\mathbf{w}$, an $n \times d$ input matrix $\mathbf{X}$ and an $n \times 1$ label vector $\mathbf{y}$ as inputs. The first function returns a scalar value `obj_val` that is the objective value in Eq. (1) (i.e., the value of $\frac{1}{2n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$). The second function returns a vector `gradient` that is the *analytic* gradient of size $d \times 1$. Both are evaluated at the given parameters $\mathbf{w}$.

Debug hint: You may want to compare your analytic gradient with the gradient computed by the `grad` function from [autograd](#) or [jax](#).

(a.2) (1%) Write a Python function

```
w = find_opt(obj_func, grad_func, X, y)
```

that finds the optimal solution of a convex optimization problem, using the `minimize` from `scipy.optimize`. The function takes an objective function `obj_func` (that admits the I/O as in part (a.1)), a gradient function `grad_func` (that admits the I/O as in part (a.1)), an $n \times d$ input matrix $\mathbf{X}$ and an $n \times 1$ label vector $\mathbf{y}$ as inputs. It returns the final parameter vector $\mathbf{w}$ of size $d \times 1$. Specifically, it should work as follows

```
def find_opt(obj_func, grad_func, X, y):
    d = X.shape[1]
    w_0 = # TODO: Initialize a random 1-D array of parameters of size d
    # TODO: Define an objective function `func` that takes a single argument (w)
    # TODO: Define a gradient function `gd` that takes a single argument (w)
    return minimize(func, w_0, jac=gd)['x'][:, None]
```

Hint: Although in machine learning, we usually represent vectors as column vectors (for example, we write $\mathbf{w}$ as a $d \times 1$ vector), some libraries work with row vectors (1-D array) by default. In the case of `scipy.optimize.minimize`, the function `func` and the gradient (i.e., Jacobian with only one output) `gd` have to take a **single 1-D array** as input. Since our `obj_func` and `grad_func` require column vectors as inputs, you would need to do some conversion in your `func` and `gd`. This is also the reason why we have `[:, None]` at the end, which converts the 1-D optimal solution to a column vector.

Debug hint: You can pass the objective and gradient functions from Q2(a) to your `find_opt` function:

```
w = find_opt(linearRegL2Obj, linearRegL2Grad, X, y)
```

You should get a solution $\mathbf{w}$ very close to your analytic solution from Q1(a) for linear regression problems. If so, then it is very likely that your `find_opt` function is correct.

(b) (2%) Solving Logistic Regression

Similar to (a), now we want to implement the objective and gradient functions for logistic regression. Recall that the objective function of logistic regression is

$$J(\mathbf{w}) = \frac{1}{n} [-\mathbf{y}^\top \log(\sigma(X\mathbf{w})) - (\mathbf{1}_n - \mathbf{y})^\top \log(\mathbf{1}_n - \sigma(X\mathbf{w}))] \quad (3)$$

where the sigmoid function σ is applied element-wise, and its gradient is

$$\nabla J(\mathbf{w}) = \frac{1}{n} X^\top (\sigma(X\mathbf{w}) - \mathbf{y}). \quad (4)$$

Numerical issue: Note that the sigmoid function σ can produce a floating-point number that is very close to zero, or exactly zero due to underflow. In such cases, $\log(0)$ will produce an `NaN`. To avoid this, a numerically stable implementation is required, so we resort to the `numpy.logaddexp` function. In general, you need to be careful whenever `exp` or `log` is called.

(b.1) (0.5%) Show, mathematically step-by-step (in the PDF report), that

$$-\log(\sigma(X\mathbf{w})) = \text{np.logaddexp}(0, -X\mathbf{w}). \quad (5)$$

Hint: Use the definitions of σ and the `np.logaddexp` function. The properties of `log` may also be helpful.

(b.2) (0.5%) Similarly, show, mathematically step-by-step, that

$$-\log(\mathbf{1}_n - \sigma(X\mathbf{w})) = \text{np.logaddexp}(X\mathbf{w}, 0). \quad (6)$$

(b.3) (1%) Now with the derivation above, implement two Python functions

```
obj_val = logisticRegObj(w, X, y)
grad = logisticRegGrad(w, X, y)
```

that take a $d \times 1$ vector of parameters $\mathbf{w}$, an $n \times d$ input matrix $\mathbf{X}$ and an $n \times 1$ label vector $\mathbf{y}$. The first function returns a scalar value `obj_val` that is the objective value (cross-entropy loss) of Equation (3), while the second one returns a vector `gradient` that is the *analytic* gradient of size $d \times 1$ as in Equation (4).

Debug hint: You may want to compare your analytic gradient with the gradient computed by the `grad` function from `autograd` or `jax`.

Debug Hint: Once done, you can use `A1testbed.py` to visualize your models and make sure that they are reasonable.

(c) (2%) Synthetic Binary Classification Problem

(c.1) (1%) In this part, you will evaluate your implementation on a synthetic dataset. Implement a Python function

```
train_acc, test_acc = synClsExperiments()
```

that returns a 4×3 matrix `train_acc` of average training accuracies and a 4×3 matrix `test_acc` of average test accuracies (see Table 5 and Table 6 below). It repeats 50 runs as follows

```
def synClsExperiments():

    def genData(n_points, dim1, dim2):
        """
        This function generates synthetic data
        """
        c0 = np.ones([1, dim1]) # class 0 center
```

```

c1 = -np.ones([1, dim1]) # class 1 center
X0 = np.random.randn(n_points, dim1 + dim2) # class 0 input
X0[:, :dim1] += c0
X1 = np.random.randn(n_points, dim1 + dim2) # class 1 input
X1[:, :dim1] += c1
X = np.concatenate((X0, X1), axis=0)
X = np.concatenate((np.ones((2 * n_points, 1))), X), axis=1 # augmentation
y = np.concatenate([np.zeros([n_points, 1]), np.ones([n_points, 1])], axis=0)
return X, y

def runClsExp(m=100, dim1=2, dim2=2):
    '''
    Run classification experiment with the specified arguments
    '''

    n_test = 1000
    Xtrain, ytrain = genData(m, dim1, dim2)
    Xtest, ytest = genData(n_test, dim1, dim2)

    w_logit = find_opt(logisticRegObj, logisticRegGrad, Xtrain, ytrain)
    ytrain_hat = # TODO: Compute predicted labels of the training points
    train_acc = # TODO: Compute the accuracy of the training set

    ytest_hat = # TODO: Compute predicted labels of the test points
    test_acc = # TODO: Compute the accuracy of the test set

    return train_acc, test_acc

n_runs = 50
train_acc = np.zeros([n_runs, 4, 3])
test_acc = np.zeros([n_runs, 4, 3])

# TODO: Change the following random seed to one of your student IDs
np.random.seed(42)

for r in range(n_runs):
    for i, m in enumerate((10, 50, 100, 200)):
        train_acc[r, i, 0], test_acc[r, i, 0] = runClsExp(m=m)
    for i, dim1 in enumerate((1, 2, 4, 8)):
        train_acc[r, i, 1], test_acc[r, i, 1] = runClsExp(dim1=dim1)
    for i, dim2 in enumerate((1, 2, 4, 8)):
        train_acc[r, i, 2], test_acc[r, i, 2] = runClsExp(dim2=dim2)

# TODO: compute the average accuracies over runs
# TODO: return a 4-by-3 training accuracy variable and a 4-by-3 test accuracy variable

```

In the PDF report, show the *averages* (over 50 runs) for each accuracy in the two tables below (one for training and the other for test).

Table 5: Training accuracies with different hyper-parameters

m	Train Accuracy	dim1	Train Accuracy	dim2	Train Accuracy
10		1		1	
50		2		2	
100		4		4	
200		8		8	

Table 6: Test accuracies with different hyper-parameters

m	Test Accuracy	dim1	Test Accuracy	dim2	Test Accuracy
10		1		1	
50		2		2	
100		4		4	
200		8		8	

Debug Hint: For your verification, we include some toy data in the `toy_data` folder. If you train with the `classification_train.csv` (with augmentation), you should get a model close to $\mathbf{w} = [0.0318, -2.47, -2.45]^\top$ and the training accuracy on `classification_train.csv` and test accuracy on `classification_test.csv` should be 0.93 and 0.925, respectively.

(c.2) (1%) Looking at your tables from above, analyze the results and discuss any findings you may have and the possible reason behind them.

(d) (1.5%) Real-World Binary Classification Problem

(d.1) (1%) Now you will apply logistic regression to a real-world problem using the [Breast Cancer Wisconsin](#) (BCW) dataset (click to see the data website).

To start, you need to preprocess the data. Implement a Python function

```
X, y = preprocessBCW(dataset_folder)
```

that takes an absolute path (result of `os.path.abspath`) of the BCW `dataset_folder` (where the `wdbc.data` file is located) and returns the preprocessed $n \times d$ input matrix `X` and an $n \times 1$ label vector `y` (that are suitable to use for your `find_opt` learning). In this function, you need to remove the ID column and use the correct target variable, converting B to 0 and M to 1.

(d.2) (0.5%) Implement a Python function

```
train_acc, test_acc = runBCW(dataset_folder)
```

that takes the absolute dataset path and returns the training and test accuracies. It runs as follows

```
def runBCW(dataset_folder):

    X, y = preprocessBCW(dataset_folder)
    n, d = X.shape
    X = np.concatenate((np.ones((n, 1)), X), axis=1) # augment

    n_runs = 50
    train_acc = np.zeros([n_runs])
    test_acc = np.zeros([n_runs])

    # TODO: Change the following random seed to one of your student IDs
    np.random.seed(42)

    for r in range(n_runs):

        # TODO: Randomly partition the dataset into two parts (50%
        #         training and 50% test; or as evenly as possible)

        w = find_opt(logisticRegObj, logisticRegGrad, Xtrain, ytrain)

        # TODO: Evaluate the model's accuracy on the training
        #         data. Save it to `train_acc`
```

```
# TODO: Evaluate the model's accuracy on the test
#       data. Save it to `test_acc`

# TODO: compute the average accuracies over runs
# TODO: return two variables: the average training accuracy and average test accuracy
```

In the PDF file, report the *average* accuracies (over 50 runs).